

Experiencia VJ-2

0 - Prerrequisitos y el Manual

1 - Abrir el Proyecto en VS Code

1.1 - Extensiones Recomendadas

2 - Jorge vs los BUGS (pero ahora de verdad)

2.1 - Cargar los Assets (TODO)

2.2 - Inicializar Variables (TODO)

2.3 - Mover las Balas (TODO)

2.4 - Pew Pew Pew (TODO)

2.5 - Detectar el Disparo (TODO)

2.6 - Destrucción Total (TODO)

3 - A Pulir

3.1 - Rotación de las Balas (TODO)

3.2 Falta para el Entregable (EVIL TODO)

0 - Prerrequisitos y el Manual

Para el desarrollo de esta experiencia necesitan tener instalada la librería **pygame-ce** . Pueden revisar la instalación ejecutando:

```
python -m pygame.example.aliens
```

Si no les reconoce el comando **python** pueden probar con alguna de **py py3 python3**

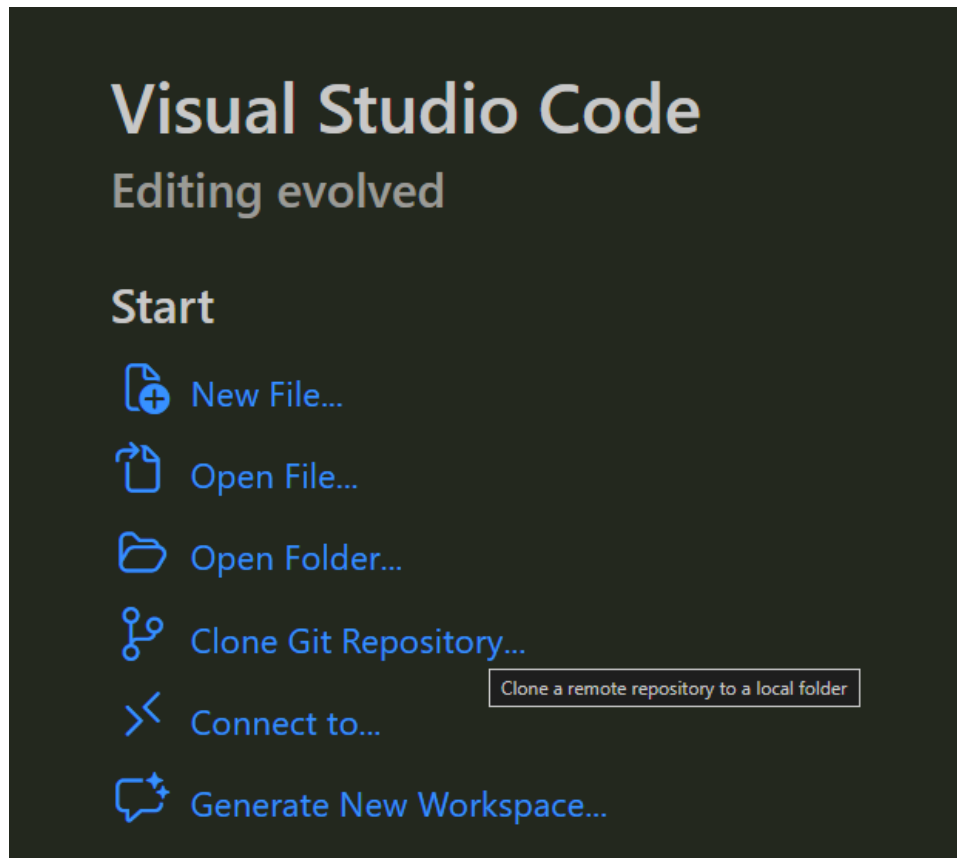
En este manual encontraras dos tipos de secciones:

- **INFO:** Contiene información y explica el funcionamiento del código base.
- **TODO:** Contiene las diversas tareas por hacer (*to-do*) para el entregable.

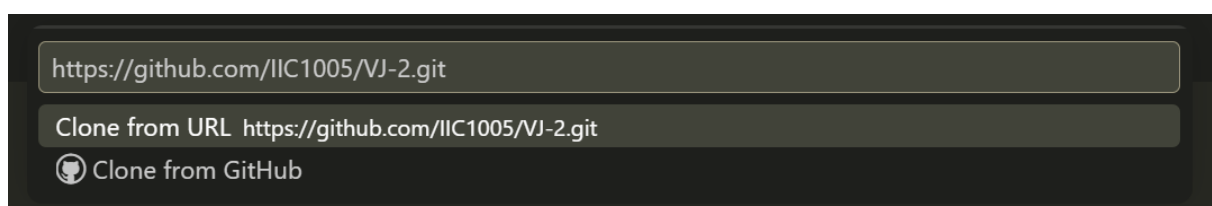
Todos los **TODO** tienen un comentario asociado en el código, indicando el lugar donde tendrán que completar.

1 - Abrir el Proyecto en VS Code

Tal como vimos en la experiencia GIT y al igual que la experiencia anterior, deben clonar el [repositorio base](#) desde GitHub.

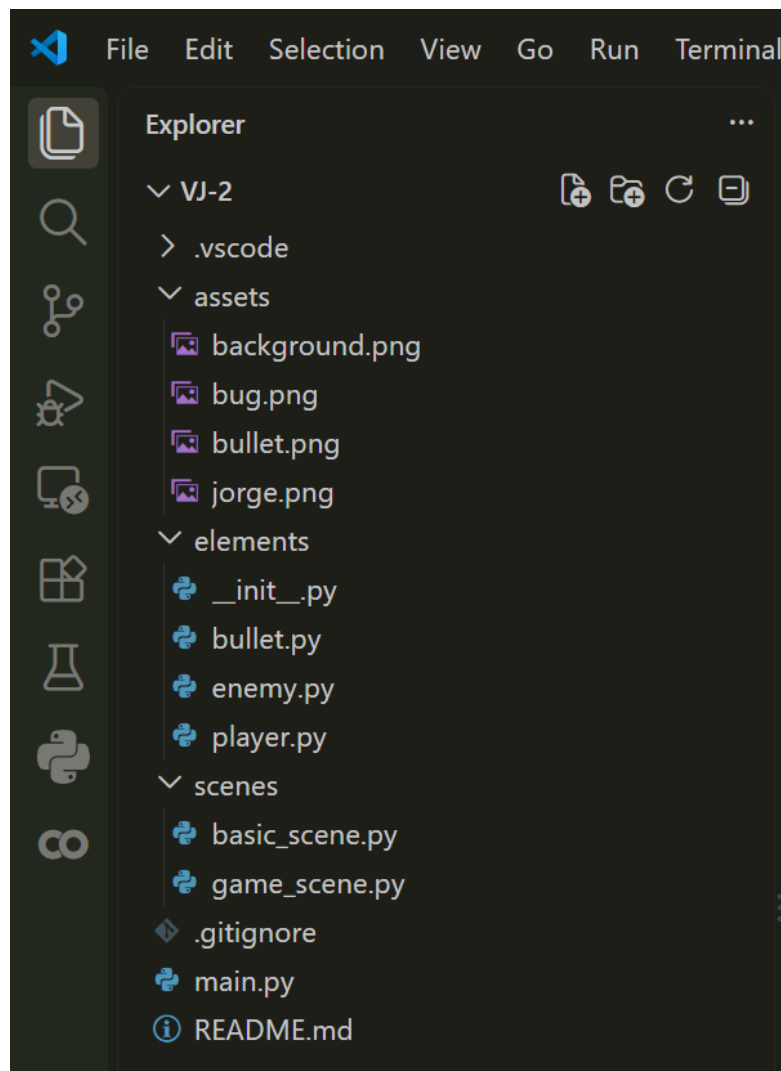


En una nueva ventana de VS Code, le damos a "Clone Git Repository..."



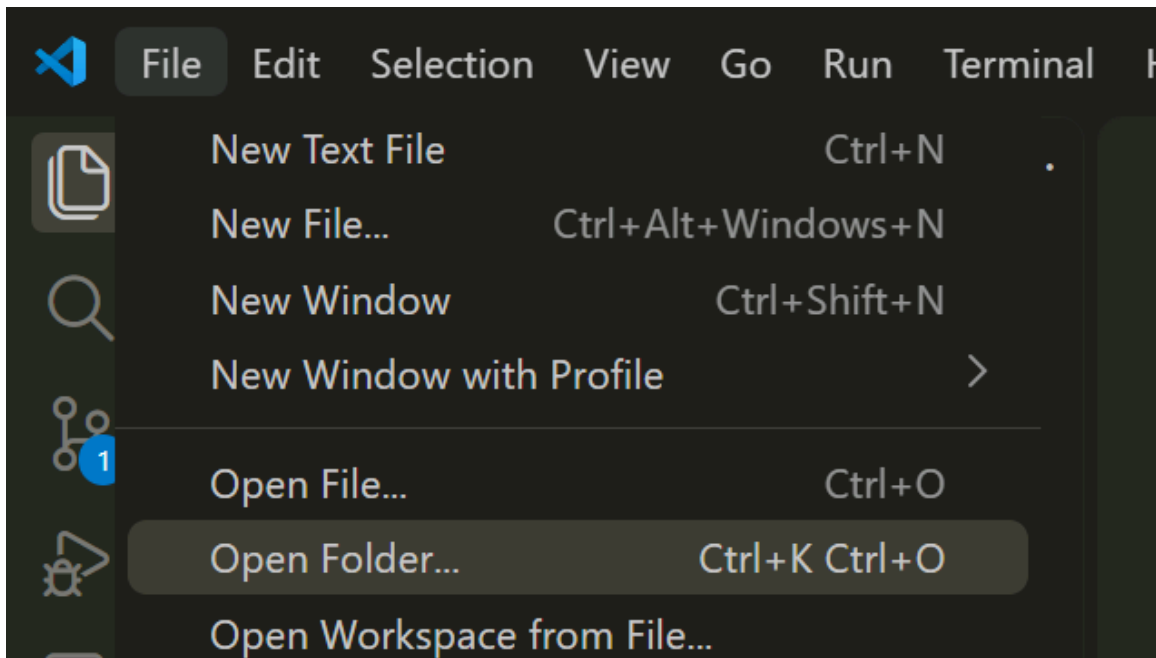
Pegamos el link y clonamos. El explorador les pedirá donde guardar el repositorio.

Una vez clonado y abierto el repositorio, deberían ver la siguiente estructura en VS Code.

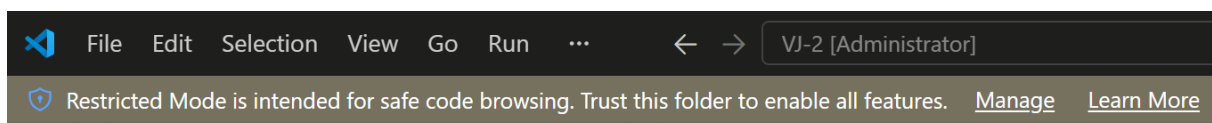


Estructura base del proyecto.

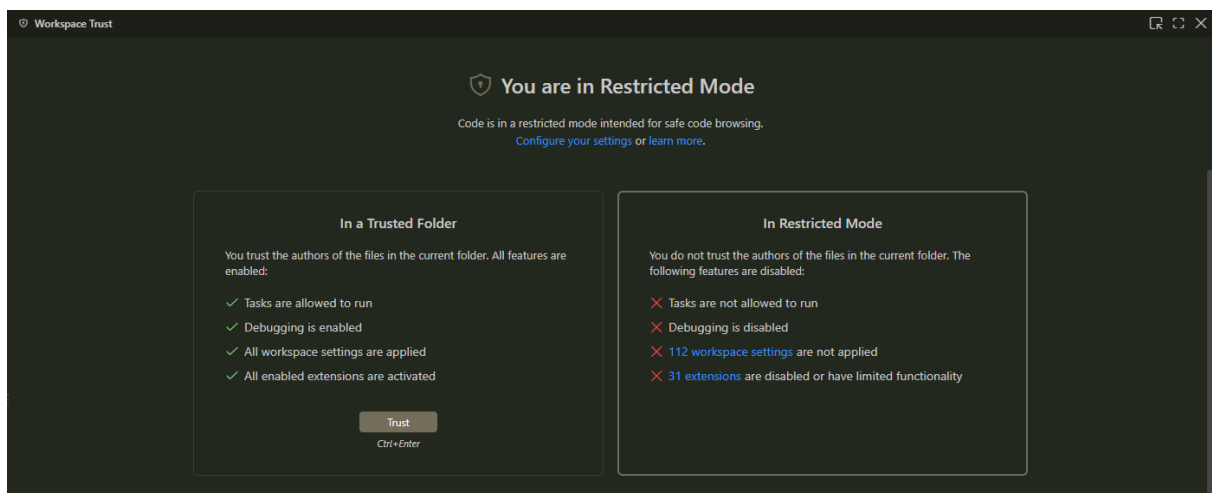
IMPORTANTE: En el explorador de VS Code, la carpeta abierta debe ser **EXACTAMENTE** "VJ-2". Si hay otra carpeta mas arriba que esta, el código **no les funcionara y les saldrán errores**. Para abrir correctamente la carpeta, pueden presionar **File** → **Open Folder** y abren "VJ-2"



IMPORTANTE: Si no les aparece el botón para ejecutar **main.py**, tienen que presionar "Manage" en la alerta que les aparece en la parte superior de VS Code y luego "Trust"



Alerta modo restringido.



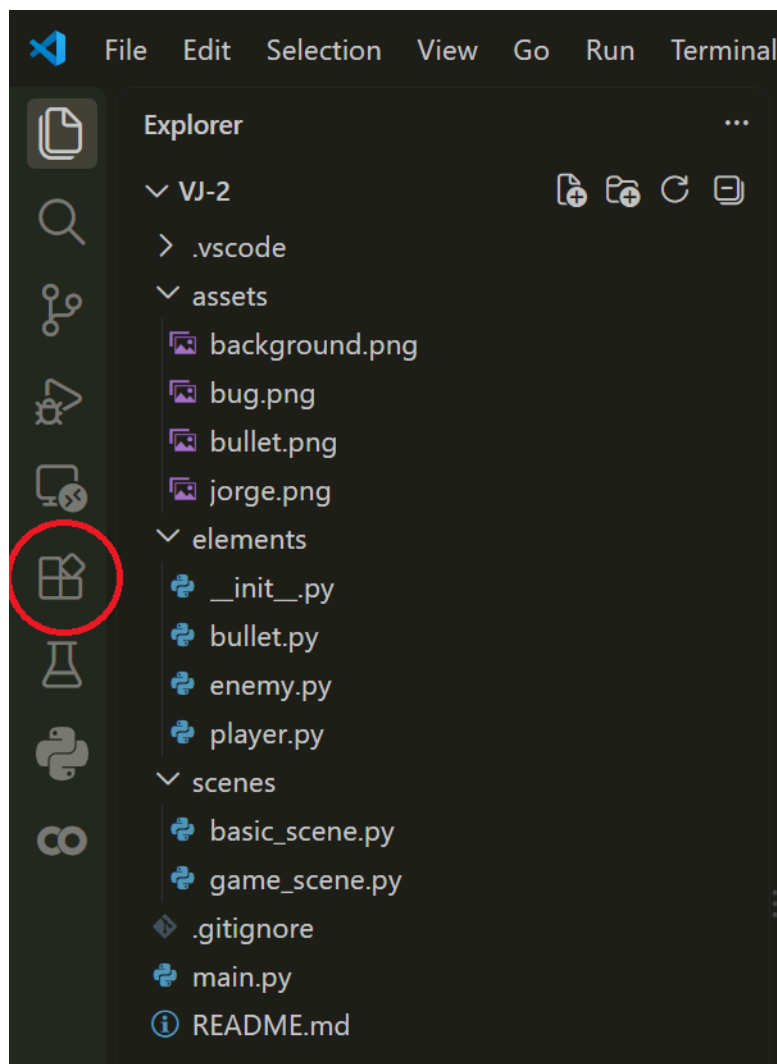
Pop-up para desactivar el modo restringido en el *workspace*.

Como pueden ver, el proyecto es muy similar a lo que ya hicieron en **VJ-1**. Para esta experiencia, vamos a **mejorar** y **agregar** nuevas funcionalidades a nuestro juego, por lo que les entregamos **VJ-1** ya resuelta.

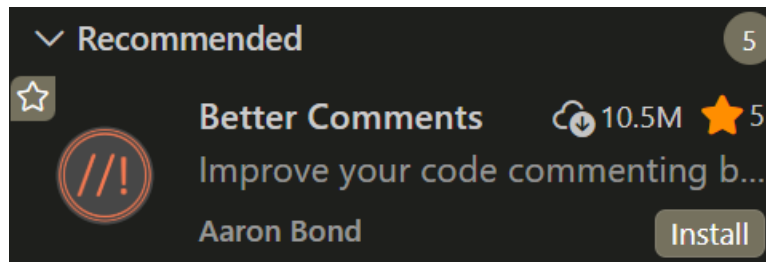
Nota: Pueden utilizar su entrega de **VJ-1** si lo desean, pero **sera su responsabilidad encontrar los nuevos assets** y no tendrán los comentarios **TODO** actualizados, por lo que se recomienda fuertemente utilizar el nuevo código base que les entregamos.

1.1 - Extensiones Recomendadas

Las siguientes extensiones deberían aparecer en la sección **"Recommended"** en el buscador de extensiones de VS Code.

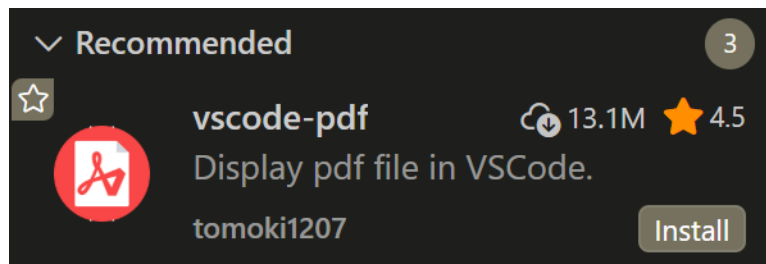


Better Comments



Permite colorear comentarios en el código y así coordinarlos con el código de color del manual.

vscode-pdf



Permite abrir archivos PDF dentro de VS Code.

2 - Jorge vs los BUGS (pero ahora de verdad)

Hasta ahora en nuestro juego nos podemos mover y esquivar a los **BUGS**, pero nos falta algo. Para que el juego no sea solo los **BUGS** atacando a **Jorge** infinitamente, vamos a darle a **Jorge** una manera de defenderse. Y todos sabemos que la mejor defensa es el ataque.

2.1 - Cargar los Assets (TODO)

Para empezar, tendremos que crear un nuevo **element** para modelar los proyectiles que va a disparar **Jorge**. Esto significa que necesitamos un nuevo **sprite** y programar su lógica. En **assets/**, ya les entregamos una imagen para la bala, **bullet.png**, y en **elements/** encontraran el archivo **bullet.py**, el cual les servirá como esqueleto para programar este nuevo objeto.

Comenzamos cargando el asset del proyectil usando **pygame.image.load** y cambiando su tamaño con **pygame.transform.scale**

```
# TODO (2.1): Cargar imagen de la bala
bulletPNG = pygame.image.load("assets/bullet.png")
bulletPNG_scaled = pygame.transform.scale(bulletPNG, (26, 14))
```

elements/bullet.py

Que hace **pygame.image.load()**? Recibe la ruta de la imagen que queremos cargar y la guarda en la variable asignada.

Que hace **pygame.transform.scale()**? Recibe la imagen que queremos ajustar y el tamaño deseado, entregándonos la imagen escalada.

Una vez cargada y escalada nuestra imagen, la usamos para definir los atributos **self.image** y **self.rect**

```
# TODO (2.1): Aspecto inicial de nuestra bala
self.image = bulletPNG_scaled
self.rect = self.image.get_rect()
```

elements/bullet.py

2.2 - Inicializar Variables (TODO)

Para que nuestra bala se comporte de forma correcta, va a necesitar de cierta información.

- En que **posición** debe aparecer?
- En que **dirección** se debe mover?
- Cuales son las **dimensiones** de la pantalla?

Toda esta información la define **Jorge** al momento de disparar y **no la bala**, por lo que se la debemos pasar al momento de crearla.

Entonces, agregamos los siguientes **parámetros** al método **__init__** de la clase **Bullet**

```
class Bullet(pygame.sprite.Sprite):
    # TODO (2.2): Agregar parametros al constructor
    def __init__(self, start_pos, direction, screen_width, screen_height):

        # ? super().__init__() inicializa la clase padre (Sprite)
        super().__init__()
```

elements/bullet.py

Nota: no es necesario modificar la llamada a **super().__init__()**

Le pasamos la posición inicial como centro a **self.rect**

```
# TODO (2.2): Posicionar la bala en la posición inicial
self.rect.center = start_pos
```

elements/bullet.py

y por ultimo, no debemos olvidar guardar la información para poder utilizarla mas adelante. Además, definimos la velocidad de la bala.

```
# TODO (2.2): Variables requeridas por nuestra bala
self.speed = 20
self.direction = direction
self.screen_width = screen_width
self.screen_height = screen_height
```

elements/bullet.py

Acá, **self.direction** va a ser del tipo **pygame.math.Vector2**, equivalente a una tupla de dos dimensiones (x, y), pero que nos permitirá hacer operaciones vectoriales fácilmente, lo cual sera relevante un poco mas adelante.

2.3 - Mover las Balas (TODO)

Ahora, tenemos que programar el movimiento de las balas. Para esto, modificamos el método **update** de nuestra clase.


```
def update(self):
    # TODO (2.3): Mover la bala
    self.rect.move_ip(
        self.direction.x * self.speed,
        self.direction.y * self.speed,
    )
```

elements/bullet.py

Que hace **rect.move_ip()**? Recibe dos valores, pixeles a mover en el eje X y pixeles a mover en el eje Y. Luego, moverá el **rect** (y su image asociada) en esa cantidad.

No hay que olvidar destruir las balas cuando se escapan de los límites de la pantalla.

```
# TODO (2.3): Eliminar la bala si sale de la pantalla
if (
    self.rect.right < 0
    or self.rect.left > self.screen_width
    or self.rect.bottom < 0
    or self.rect.top > self.screen_height
):
    self.kill()
```

elements/bullet.py

Con esto, tenemos lista toda la lógica de nuestra bala. Ahora, tenemos que darle el blaster a **Jorge**.

2.4 - Pew Pew Pew (TODO)

Como **Jorge** es quien dispara, le vamos a dar a él la responsabilidad de **crear**, **guardar** y **actualizar** las balas. En primer lugar, vamos a crear un nuevo **Group** para almacenar todos los proyectiles existentes.

```
# TODO (2.4): Crear grupo de balas
self.bullets = pygame.sprite.Group()
```

elements/player.py

Si recuerdan de **VJ-1**, los **grupos** hacen que sea muy fácil manejar y **actualizar** todos los elementos de este de forma simultanea.

Ya que Jorge es el dueño de este nuevo **grupo**, simplemente podemos actualizarlo cada vez que **Jorge** se actualice. Entonces, en el método update de Jorge agregamos

```
# TODO (2.4): Actualizar las balas
self.bullets.update()
```

elements/player.py

Por ultimo, tenemos que crear las balas cada vez que se haga click en la pantalla, apuntando hacia donde se encuentre el mouse. Para esto, vamos a definir un nuevo método **shoot**.

Este método recibe la posición del mouse, calcula la dirección en la que la bala debe moverse por ultimo, crea la bala y la agrega al grupo.

```
def shoot(self, mouse_pos):
    # TODO (2.4): Calcular direccion de la bala
    distance = Vector2(mouse_pos) - Vector2(self.rect.center)
    direction = distance.normalize()
```

elements/player.py

Acá, utilizamos la clase **Vector2** de **pygame.math** para calcular de forma fácil dos cosas:

- Primero, calculamos la **distancia** entre la **posición objetivo** (dada por las coordenadas del mouse) y el **origen de la bala** (dado por el centro de nuestro Player). El resultado sera un **Vector2** con la distancia en el eje X y la distancia en Y.

- Luego, si normalizamos la distancia, obtenemos un **Vector2** unitario que nos indicará los **componentes de dirección** en cada eje. De esta forma, todas nuestras balas se moverán a la misma velocidad independientemente de la distancia entre el jugador y a donde apuntamos.

Por si no lo han notado, incluso al programar juegos, no nos podemos escapar de las matemáticas, especialmente si se trata de **Algebra Lineal**. Si quisieran, podrían hacer todos estos cálculos con operaciones matemáticas base de **python**, pero usamos la implementación de **pygame** para que sea mas rápido y sencillo.

Una vez definida la dirección, creamos la instancia de **Bullet** y la agregamos al **Group**.

```
# TODO (2.4): Crear bala y agregarla al grupo de balas
bullet = Bullet(
    self.rect.center,
    direction,
    self.screen_width,
    self.screen_height,
)
self.bullets.add(bullet)
```

elements/player.py

2.5 - Detectar el Disparo (TODO)

Jorge ya tiene munición y un arma preparada, solo nos falta de conectar el click del mouse con el disparo. Para esto, vamos a detectar el evento **MOUSEBUTTONDOWN** en nuestro **gameloop**.

```
# TODO (2.5): Disparar balas al hacer click con el mouse
elif event.type == MOUSEBUTTONDOWN:
    mouse_pos = pygame.mouse.get_pos()
    player.shoot(mouse_pos)
```

scenes/game_scene.py

Que hace `pygame.mouse.get_pos()`? Nos entrega las coordenadas X e Y de del mouse dentro de la ventana del juego.

Ahora que podemos crear balas, tenemos que dibujarlas.

```
# TODO (2.5): Dibujar las balas en la ventana
for bullet in player.bullets:
    screen.blit(bullet.image, bullet.rect)
```

scenes/game_scene.py

Que hace `.blit()`? Toma dos argumentos, el **QUE** se dibujara y el **DONDE** se dibujara en la superficie que escojamos, en este caso, `screen`. Recordar que ya definimos el centro de `bullet.rect` como el origen de la bala.

2.6 - Destrucción Total (TODO)

Por ultimo, tenemos que darle **efectividad** a las balas, por lo que hay que detectar **colisiones**. En la experiencia anterior, ya utilizaron `spritecollideany`, que detecta colisiones entre un **sprite** y un **grupo**. En este caso no nos sirve ya que necesitamos detectar colisiones entre dos **grupos**, **enemies** y **bullets**. Para esto vamos a utilizar `pygame.sprite.groupcollide`.

```
# TODO (2.6): Calcular colisiones entre balas y enemigos
pygame.sprite.groupcollide(
    player.bullets,
    enemies,
    True,
    True,
)
```

scenes/game_scene.py

Este detector de colisiones recibe 4 argumentos

- **groupa**: El primer grupo (A) sobre el cual queremos detectar colisiones
- **groupb**: El segundo grupo (B) sobre el cual queremos detectar colisiones
- **dokilla**: Si queremos eliminar el elemento del grupo A que colisionó

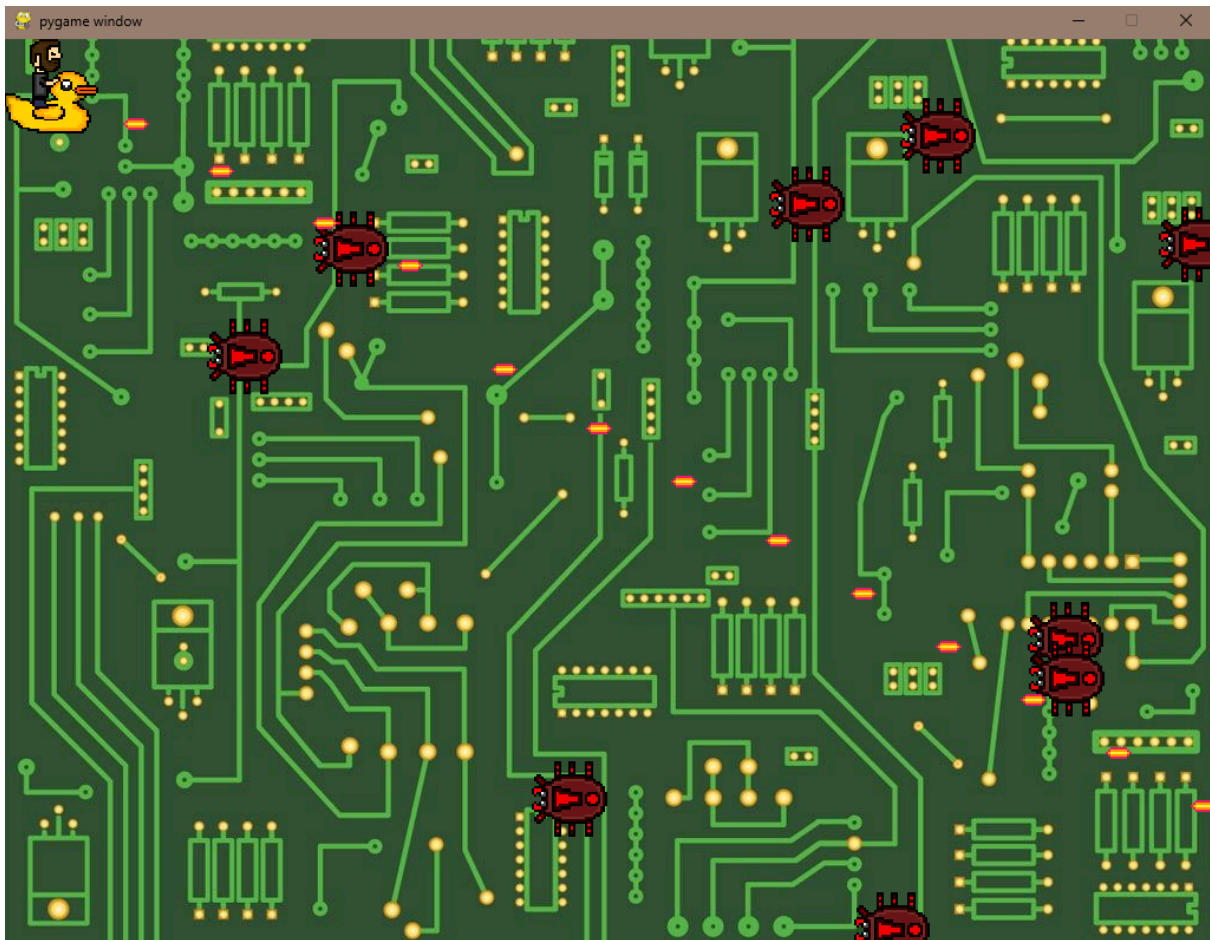
- **dokillb**: Si queremos eliminar el elemento del grupo B que colisionó

En este caso, queremos eliminar tanto el enemigo como la bala, de aquí el doble **True**.

3 - A Pulir

Ya han programado la primera feature!

Si ejecutan **main.py**, ya van a poder disparar y eliminar BUGS al hacer click!



Pero hay algo medio raro no? La imagen de la bala siempre apunta en la misma dirección sin importar a donde disparemos. Esto no afecta el funcionamiento del juego, pero se ve incorrecto.

Con esto, pasamos al siguiente tema y desafío de la experiencia: El **Pulido**.

Pulir un juego es agregar y arreglar detalles que, a pesar de que no cambian el gameplay de este, afectan la experiencia. Es por esto que a continuación van a tener que hacer algunos cambios por su cuenta para pulir el juego.

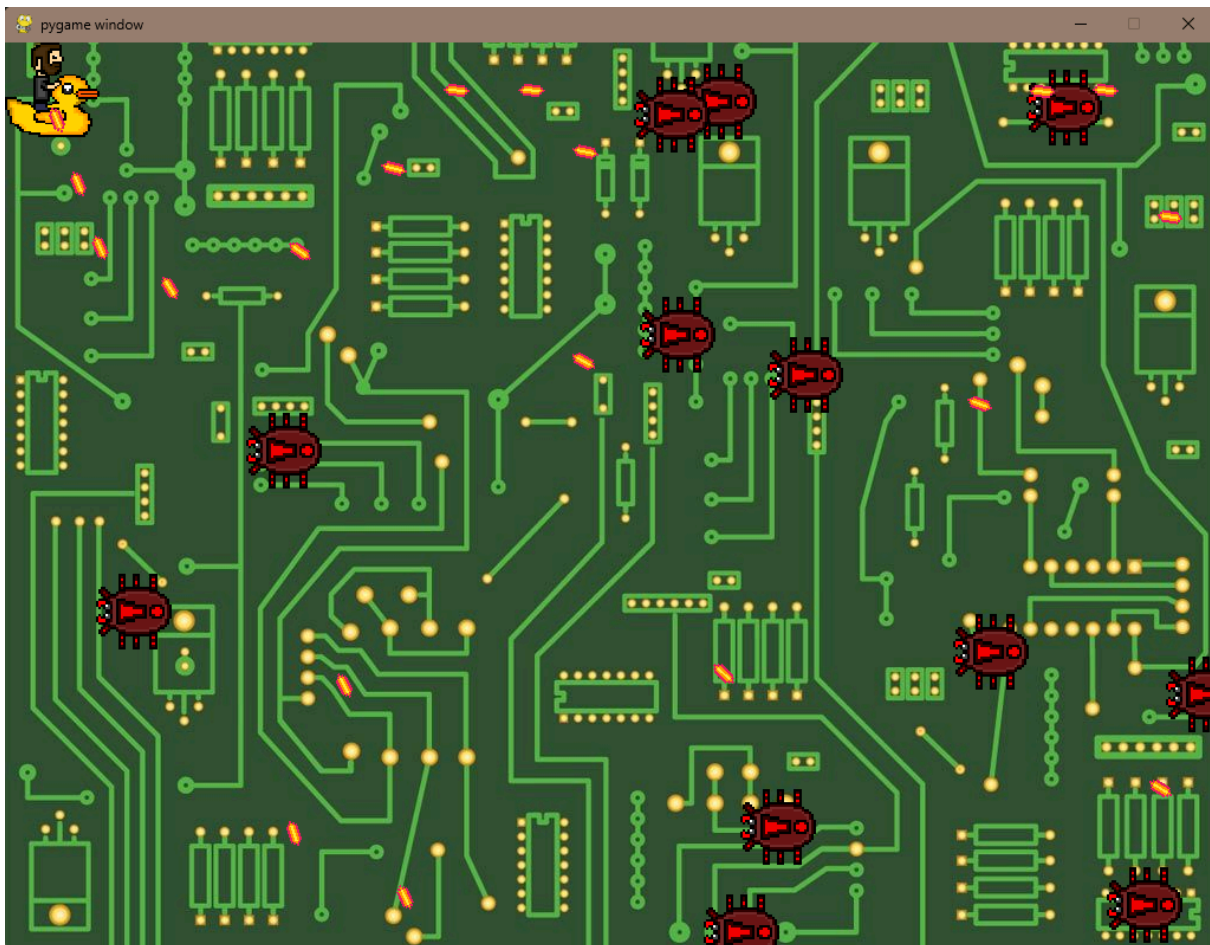
3.1 - Rotación de las Balas (TODO)

Solo para este requisito les vamos a mostrar como se hace, el resto tendrán que ser implementados por su cuenta. En **bullet.py** agregamos lo siguiente:

```
# TODO (3.1): Rotar la bala para que apunte en la dirección correcta
angle = self.direction.angle_to(Vector2(1, 0))
self.image = pygame.transform.rotate(bulletPNG_scaled, angle)
```

elements/bullet.py

Primero, calculamos el **angulo** entre la dirección de la **bala** y un **Vector2** horizontal de referencia, para luego **rotar** la imagen según este angulo con **pygame.transform.rotate**. Con solo estas dos líneas de código, el juego ya se ve un poco mejor.



3.2 Falta para el Entregable (EVIL TODO)

Ahora, implementa de manera autónoma las siguientes features para pulir aun mas el juego.

- Agregar una **mirilla** (algo que **SIGA** la posición del **MOUSE**, busquen alguna imagen en google en formato **png** para que tenga fondo transparente)
- Crear una **pantalla de muerte** (debe permitir **SALIR** o **VOLVER A JUGAR**)
- Algún tipo de **cooldown** para el disparo (por ejemplo, disparar una vez y tener que **esperar x tiempo** o **recargar** después de x disparos)

Esta sección **no tiene comentarios** en el código. Parte del desafío es saber donde implementar cada cosa. Recuerden que pueden utilizar **Google** y la **IA** de forma razonable, ademas de la [documentación de pygame](#).

El nivel de pulido sera muy valorado en VJ HONORS!!!